

CS 281 Systems Architecture Syllabus

Instructor Information

Dr. Brian Mitchell

Office 1134

email: bmitchell@drexel.edu

Preferred Communication Channels: Discord for general questions, Discord DM for personal questions, Email for emergencies or high priority notifications.

Office Hours: Will be posted on course blackboard page

Course Description

This course covers internal function and organization of digital computers, including instruction set design, machine and assembly language, computer arithmetic, ALU design, central processor organization and implementation.

Course Objective and Goals

1. To gain an appreciation for why understanding how a machine works is important for a modern computer scientist especially with the emerging trends of embedded and IoT devices
2. To obtain an understanding of how a computer is organized and how it works. To develop a model of how a program executes on a computer.
3. To be able to understand an assembly language program. There will be some assignments involving assembly language programming; however, the objective is to understand the instruction set of a machine and how a program executes on a computer, rather than to be able to write full length assembly language programs.
4. To understand how a computer can be implemented (down to the gate level). In the lab associated with the course, students will implement a subset of the RISC-V architecture using the Logisim evolution digital simulator.

Prerequisites (all Min Grade: D)

(CS 270 or ECE 200) and (CS 172 or CS 176 or ECEC 301 or ECEC 201 or ECE 105) **Instructor**

Teaching Assistants - Office & Lab Hours

Contact information for course teaching assistants, including office and lab hour coverage will be posted on the course Blackboard site.

What Students Should Know Prior to this Course

1. Should be familiar with Boolean expressions, truth tables, normal forms.
2. Should be able to design a simple logic circuit.

3. Should be familiar with basic components of combinational logic: encoders, decoders, and multiplexors.
4. Should be familiar with elements of sequential logic: latches, flip flops, registers, memory.
5. Should be able to understand and design a finite state machine.
6. Should have solid programming experience.
7. Must be comfortable with the basic programming constructs in C/C++.
8. Must be comfortable with recursion and pointers.
9. Knowledge and the ability to use data structures such as arrays and lists.

What Students will be able to do upon Successfully Completing this Course Statement of Expected Learning

1. Understand what a compiler, interpreter, assembler, linker and loader does.
2. Understand the components and format of a machine instruction set.
3. Write a simple assembly language program.
4. Understand how an assembly language program executes on a computer.
5. Understand how a computer represents numbers and performs arithmetic.
6. Build a simple ALU.
7. Understand the Datapath and control of a simple computer.
8. Implement a simple instruction set: create an appropriate datapath and describe the control using microcode or a finite state machine.
9. Describe and simulate a processor using modern simulation software.

Textbook

1. Computer Organization and Design RISC-V Edition: The Hardware Software Interface (The Morgan Kaufmann Series in Computer Architecture and Design). ISBN: [0128122757](https://learning.oreilly.com/library/view/computer-organization-and/9780128122761/). This is the first edition of the textbook **and its available for free** via Drexel's library. You need to go to the library page and setup a cross-linked account with O'Reilly learning. The book is at: <https://learning.oreilly.com/library/view/computer-organization-and/9780128122761/>
2. An introduction to Assembly with RISC-V. Online / open source: <https://riscv-programming.org/book/riscv-book.html>

Topics

1. Computer Abstractions (Chapter 1)
2. Review of Digital Circuits and Logic Design (Appendix A)
3. History of Computers (Chapter 1)
4. Instructions: Language of the Machine (Chapter 2)
5. Assembly Language Programming (Chapter 2 and Supplemental Text)

6. Assemblers, Linkers, and the Venus Simulator (Class and Lab Notes)
7. Computer Arithmetic (Chapter 3)
8. The Processor: Datapath and Control (Chapter 4)
9. Hardware simulation – Logisim Evolution and Digital (Lab Notes)

Software (this will be the focus of Lab 1 – Setting up our machines)

RISC-V has very good open-source software for writing RISC-V programs using a simulator to promote learning. In this class we will be using the following tools for labs:

1. VSCode – This will be our default IDE for the programming labs. You can download it for free from here: <https://code.visualstudio.com/>
2. RISC-V VSCode Plugins:
 - a. The “RISC-V Support” plugin – provides syntax highlighting and editor awareness for RISC-V instructions
 - b. The “RISC-V Venus Simulator” plugin – provides a simulator to assemble, debug and run RISC-V programs on Windows, Mac and Linux computers.
3. **NOTE on the good and bad of using VSCode in our labs:** The goal of this class is learning the RISC-V environment, and while the use of simulators integrated as VS-Code plugins makes the process easier, additional learning is possible if you want to stretch yourself to use a real (or at least as real as possible) RISC-V environment. For those who want to challenge yourself I will also be showing you how to run a real version of RISC-V Ubuntu Linux inside of Docker in our Lab sessions. This will have the entire development toolchain including vim, ssh support, gcc, gdb, and a RISC-V assembler. Extra credit will be provided if you do the lab assignments using native tooling versus using the VSCode simulator. If you are interested in this, please ensure your Mac or Windows laptop is configured to run Docker. Windows: <https://docs.docker.com/desktop/install/windows-install/>. For Mac I really like orbstack as my docker runtime: <https://orbstack.dev/>
4. Logisim Evolution – This will be our digital hardware design and simulator platform. You can download it from here: <https://github.com/logisim-evolution/logisim-evolution>
Note previous versions of this class covered programing in hardware description languages (e.g., VHDL). In this class we will focus one of our labs on taking a logisim design and generating VHDL, along with a discussion on how to use VHDL to produce a FPGA (Field Programmable Gate Array)
5. Online Tools (used to complement lectures):
 - a. RISC-V Online Assembler: <https://riscvasm.lucasteske.dev/>
 - b. Compiler Explorer: <https://godbolt.org/>

Note that we will be covering how to download and configure these tools in our lab sessions. Lectures will include demonstrations with these tools

Grading and Policies

1. Written Assignments (seven at 5% each)	35%
2. Labs (seven at 5% each)	35%
3. Quizzes – There will be 5 online quizzes (25 pts each) in this course. Final quiz grade will be calculated: (GRADE : top 4 quiz scores @ 25 pts each) + (Extra Credit : 0.5 of your lowest quiz score). Thus your maximum score can be 112.5%.	30%
TOTAL	100%

Final grades

A+	98 – 100	C+	77 – 79
A	93 – 97	C	73 – 76
A-	90 – 92	C-	70 – 72
B+	87 – 89	D+	66 – 69
B	83 – 86	D	60 – 65
B-	80 – 82	F	< 60

The university's Academic Honesty policy is in effect for this course. Please read Drexel University Student Handbook found at <http://www.drexel.edu/Studentlife/>. On the first incident, students who share their work (even with best intentions) or otherwise violate the course or university academic honesty policy may receive a grade of F for the course (the students may not withdraw in this case). The students may be reported to the department, college, and/or University Judicial (Honesty) Board. Both the giver and the receiver will receive these penalties.

Submitting Assignments

Assignments will be submitted through BBLearn according to the directions given on the assignment page, no later than the due date and time listed on each assignment and/or the assignment page. Grade breakdowns, rubrics, and/or point valuations will be provided on each assignment as it is assigned, as appropriate. Grades will be reported via BBLearn.

Meetings of this course are recorded

Any recordings will be available to students registered for this class. Students are expected to follow appropriate university policies and maintain the security of passwords used to access recorded lectures. Recordings, or any part of the recordings, may not be reproduced, shared with those not in the class, or uploaded to other online environments. **Tentative Course Schedule**

Notes:

1. Homework's assigned during a week will be due on Monday the following week
2. Labs are designed so that 90-100% of the lab can be completed in the lab itself. The deadline for the lab deliverables is midnight on Friday the week after the lab.
3. Online quizzes will be due midnight on Wednesdays or as per the schedule in the blackboard. Note that the late day bank cannot be used for online quizzes. If you fail to submit the quiz by the deadline the grade will be a zero.
4. All deadlines subject to change at the instructor discretion. Changes will be communicated via blackboard announcements, and discord posts.

Please keep track of deadlines using the “Calendar” feature in Blackboard. In general, Homework assignments will be due at midnight on Monday's, Labs at Midnight on Friday, and on Quiz Weeks, they will be posted online by Noon on Monday and due at Midnight on Wednesday. I will, at my discretion move dates around based on interaction with students where I feel that extra time will benefit the overall class.

Week	Lecture Contents	Labs & Assignments
1	• Syllabus Review – Class Expectations & Overview • Motivation for Systems Architecture • The RISC-V Instruction Set (Part 1) – RV Processor Overview: Registers, Memory – RV basic instructions – Math and logical operations	Lab 1 – Tooling Setup and RISC-V Assembler tutorial
2	• The RISC-V Instruction Set (Part 2) – Buffers – Structuring Programs – Assembler directives – Instruction Formats – Machine language translation – Controlling execution flow with jumps and conditional branches	HW-1: Basic RV Covering Materials from first 2 lectures Lab 2 – Basic RV Programming
3	• The RISC-V Instruction Set (Part 3) – Array Processing – Looping – Pseudo Instructions	HW-2: RV Programming Concepts – Jumps, Branching and Loops

		Lab 3 – IoT Simulation fun with Venus Simulator
4	• The RISC-V Instruction Set (Part 4) – Functions / procedures – Register calling conventions – Stack Frames – RISC-V on “real” hardware overview	HW-3: RV Functions and Procedures Lab 4 – Programming using Functions and Recursion
5	• Digital Arithmetic (Review) • Digital Logic • Combinational and Sequential Logic – Latches/ Flip-Flops – Timing • ALU Design (Start)	HW-4: Logic **no lab this week
6	• ALU Design (Finish) • Single Cycle RISC-V Implementation (Part 1)	Lab 5 – Introduction to Logisim **no homework this week
7	• Single Cycle RISC-V Implementation (Part 2)	Lab 6 – Build and Test 1-Bit ALU HW-5: Implementing RISC-V Datapath Instructions
8	• Pipelined RISC-V Implementation (Part 1) - Introduction	Lab 7 – Build and Test 8-Bit ALU with Simple Datapath and Control HW-6: Implementing RISC-V Datapath Instructions
9	• Pipelined RISC-V Implementation (Part 2) – Dealing with Hazards	HW-7: Implementing RISC-V Datapath Instructions
10	• Pipelined RISC-V Implementation (Part 3) – Branch Prediction and other topics	** No homework ** No lab

Assignments, Due Date and your Late Day Bank

This course will have assignments that will be spread out over the course as mentioned above. Given my previous experience, students tend to have different levels of readiness, especially with background in programming. I will adjust deadlines for the whole class as necessary, based on

my observation of student effort and questions being asked over Discord, and attending office hours (mine or the TAs). One off deadline extensions are not possible, but I do realize that things come up from time to time. If you go into grade center, you will notice that you have a bank of 5 late days to use over the term. Late submissions will be accepted without penalty by deducting from your late day bank. Once you exhaust your bank, late assignments will take a 50% penalty on day 1, and will not be accepted on day 2+.

Note that the late day bank is to accommodate students “when things come up” for whatever reason. This includes work deadlines, needs of other courses, sickness, unexpected travel, and so on. They should be considered as “insurance”, because once they are gone, they are gone. You do not require any permission to submit assignments late – they will be accepted without penalty, without any reason for lateness so long as late days exist in your bank.

Expectations of Timely Communication for Hardships

If, for whatever reason, you find yourself struggling with keeping up with course deliverables, it is important that you let me know right away, including your plan to get back on track. I am here to help you and am willing to be as flexible as possible. It is your responsibility to be as proactive as possible in contacting me over a hardship. My flexibility becomes very limited if you come to me weeks after one or more assignment deadlines have been missed with a hardship story expecting me to accept missing work without significant penalty, if even at all.

The process you need to follow: If you are having a personal hardship that is impacting your ability to submit course deliverables on time, contact me directly either by visiting my office hours, sending me a DM over Discord or via an email to discuss. What is most important to me is that you come to the discussion prepared with a plan to get back on track with your course responsibilities. Based on the outcome of our discussion, I will, at my discretion, add additional late days to your “late day bank” to accommodate your needs to the best of my ability and in fairness to the entire class. It is your responsibility to get yourself back on track in the course.

Holidays

The academic calendar will guide holidays that may, or may not happen, during the course. Changes to content and deliverables because of holidays will be communicated by the instructor.

Office of Disability Resources

Students requesting accommodations due to a disability at Drexel University need to present a current Accommodation Verification Letter (AVL) to faculty before accommodations can be made. AVL's are issued by the Office of Disability Resources (ODR). For additional information,

visit the ODR website at <http://www.drexel.edu/oed/disabilityResources>, or contact the Office for more information: 215-895-1401 (V), or disability@drexel.edu.

University Policies

In addition to the course policies listed on this syllabus, course assignments or course website, the following University policies are in effect:

- Academic Honesty: http://www.drexel.edu/provost/policies/academic_dishonesty.asp
- Student Life Honesty Policy from Judicial Affairs:
<http://www.drexel.edu/studentlife/judicial/honesty.html>
- Students with Disability Statement: <http://drexel.edu/oed/disabilityResources/students/>
- Course Drop Policy: http://www.drexel.edu/provost/policies/course_drop.asp

The instructor may, at his/her/their discretion, change any part of the course during the term, including assignments, grade breakdowns, due-dates, and the schedule. Such changes will be communicated to students via the course web site Announcements page in BBLearn. This page should be checked regularly and frequently for such changes and announcements. Other announcements, although rare, may include class cancellations and other urgent announcements.

Drexel Student Learning Priorities:

<http://www.drexel.edu/provost/dcae/SymposiumLearningPriorities.PDF>